



TRAFFIC LIGHT SIMULATION

A Web-Based Intersection Traffic Simulator

Project Report

Submitted by: Syeda Hareem Haider

Roll Number: F2024376258

Course: Web Technologies

Course: IT291

Section: A2

Submitted to: Sir Shehan Latif

Table of Contents

Contents

Table of Contents.....	2
1. Introduction.....	3
2. Objective.....	4
3. Tools / Technologies Used.....	5
HTML5	5
CSS3	5
JavaScript (ES6)	5
Visual Studio Code	5
Live Server Extension	5
Chrome Developer Tools	5
4. Features of the Project.....	6
5. Screenshots / Output.....	7
6. Algorithm / Flowchart.....	9
7. Code Explanation.....	11
7.1 Asset loading.....	11
7.2 Signal control.....	11
7.3 Vehicle movement and stop lines.....	11
7.4 Turning logic.....	11
7.5 Spacing between vehicles.....	12
7.6 Rendering and dashboard.....	12
8. Challenges Faced.....	13
9. GitHub Repository link:.....	14
10. Conclusion.....	15
11. References.....	16

1. Introduction

Traffic signals play an important role in controlling traffic flow at road intersections. Poor signal timing can cause congestion, delays and confusion for drivers. The purpose of this project is to create a web-based traffic light simulation that demonstrates how vehicles move through an intersection and respond to traffic signals.

This simulation was developed using **HTML, CSS and JavaScript**. It represents a four-way intersection where vehicles approach from the **North, South, East and West directions**. Different types of vehicles such as **cars, buses, trucks and bikes** are generated automatically and move according to the traffic signal status.

The simulation also includes vehicle turning behavior, lane management and an adaptive traffic signal system. The green light duration changes based on the number of vehicles waiting at a particular direction, making the simulation more realistic. Through this project, I learned how animation, timing systems and object movement can be implemented using web technologies.

2. Objective

The main objective of this project is to develop a traffic intersection simulation using basic web technologies without using any external game engine or simulation software.

The specific objectives are:

- To simulate vehicle movement from four different directions.
- To implement a traffic signal system that changes automatically.
- To adjust signal timing according to traffic density.
- To create smooth vehicle turning animations.
- To maintain safe spacing between vehicles and prevent overlapping.
- To display live information such as simulation time and vehicle counts.
- To gain practical experience with HTML Canvas and JavaScript animation.

3. Tools / Technologies Used

The following tools and technologies were used during the development of this project:

HTML5

Used to create the structure of the web page and provide the Canvas element where the simulation is displayed.

CSS3

Used to design the user interface, dashboard and overall layout of the application.

JavaScript (ES6)

Used to implement all simulation functionality, including vehicle movement, traffic signal control, animation, collision spacing and dashboard updates.

Visual Studio Code

Used as the primary code editor for writing and managing the project files.

Live Server Extension

Used to run the project locally and test changes in real time within the browser.

Chrome Developer Tools

Used for debugging errors, checking asset loading issues and monitoring the behavior of the simulation during development.

4. Features of the Project

The project includes the following features:

- A four-way traffic intersection (North, East, South, West) displayed using an HTML Canvas.
- Multiple vehicle types including cars, buses, trucks and bikes, each with its own speed.
- Continuous vehicle spawning at randomised directions and types.
- Automatic signal cycling with green and yellow phases for each of the four directions in turn.
- Live dashboard showing simulation countdown timer and the number of vehicles currently in each of the four directions.
- Traffic signals with green and yellow phases.
- Adaptive signal timing based on the number of waiting vehicles.
- Stop-line functionality that prevents vehicles from crossing red signals.
- Vehicle turning behavior at the intersection.
- Safe spacing between vehicles to avoid overlapping.
- Real-time dashboard showing traffic statistics.
- Responsive design that adjusts to different screen sizes.
- Responsive canvas sizing that keeps the intersection visible without distortion across different screen sizes.

These features help create a realistic representation of traffic movement at an intersection.

5. Screenshots / Output

The following screenshots show the simulation running in the browser.

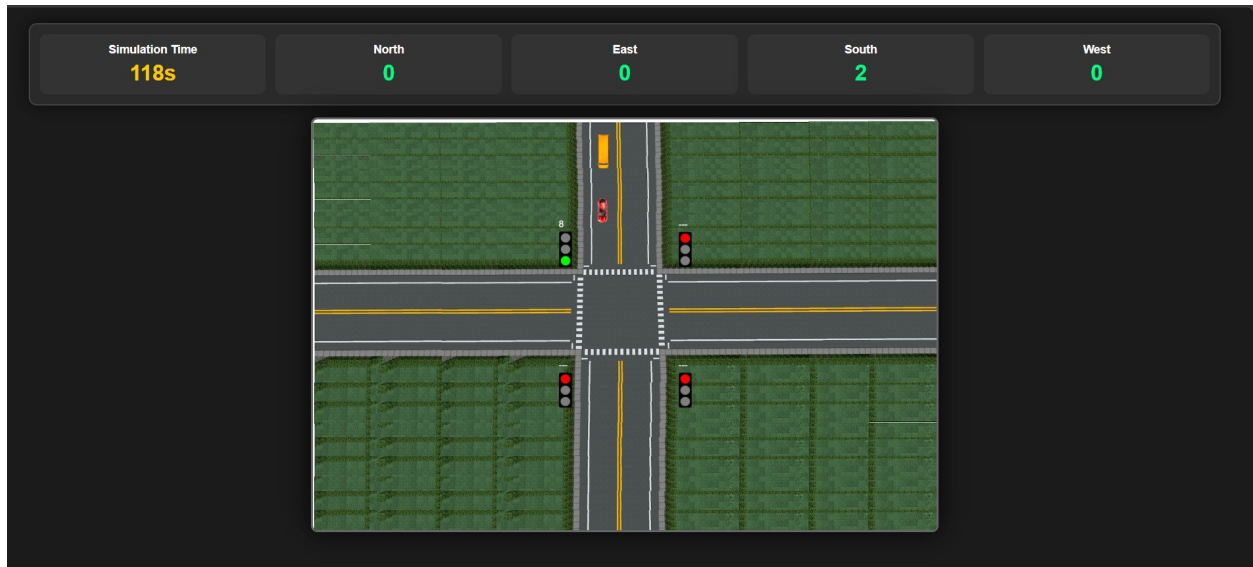


Figure 5.1: Initial view of the intersection with the dashboard

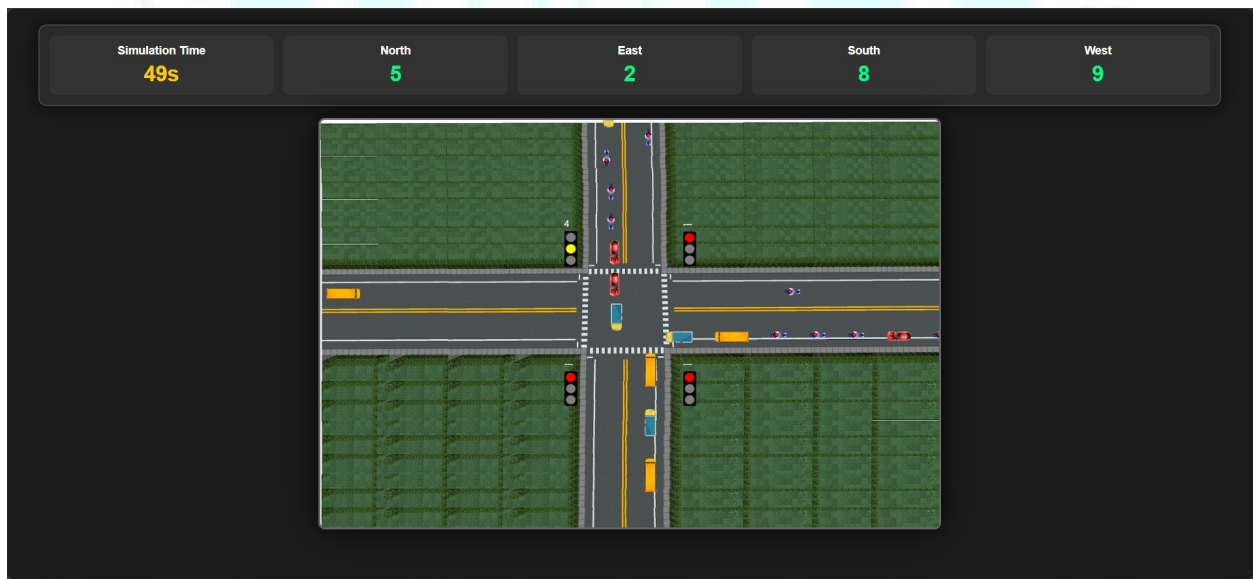


Figure 5.2: Vehicles queued at a red signal

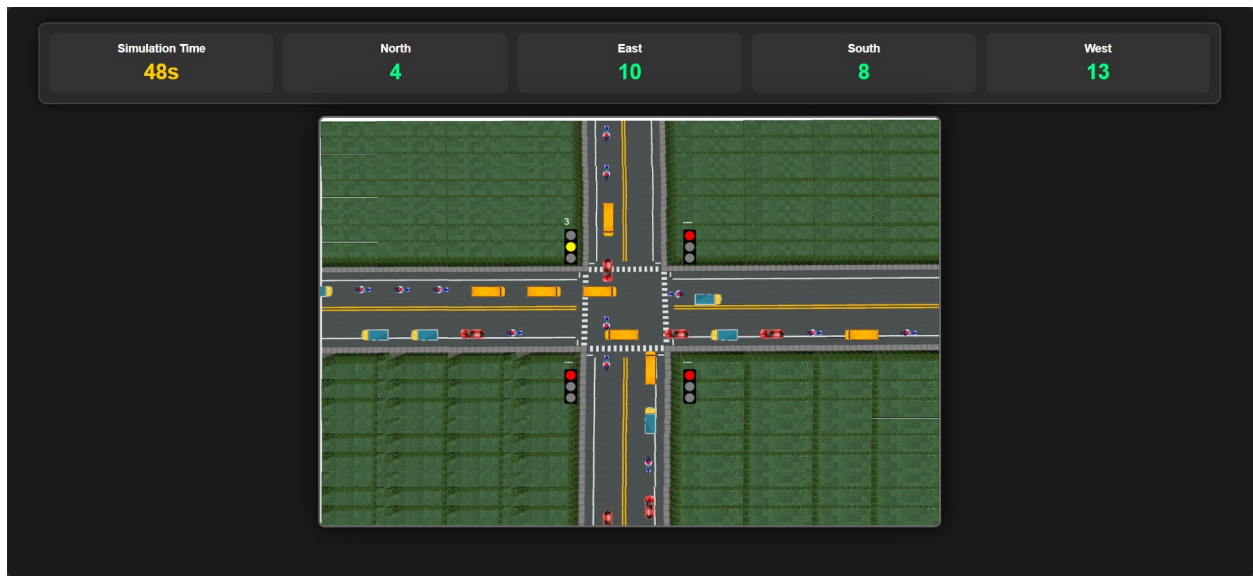


Figure 5.3: A vehicle turning at the intersection

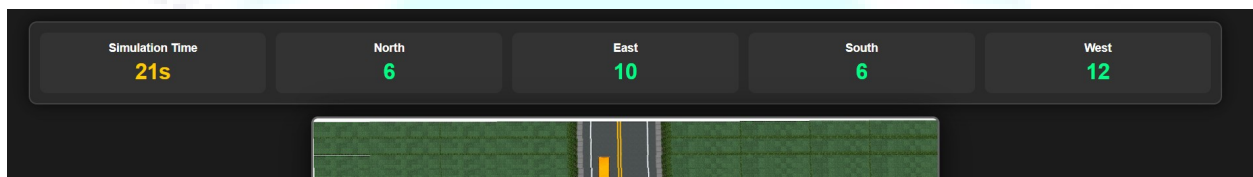


Figure 5.4: Dashboard showing live per-direction vehicle counts

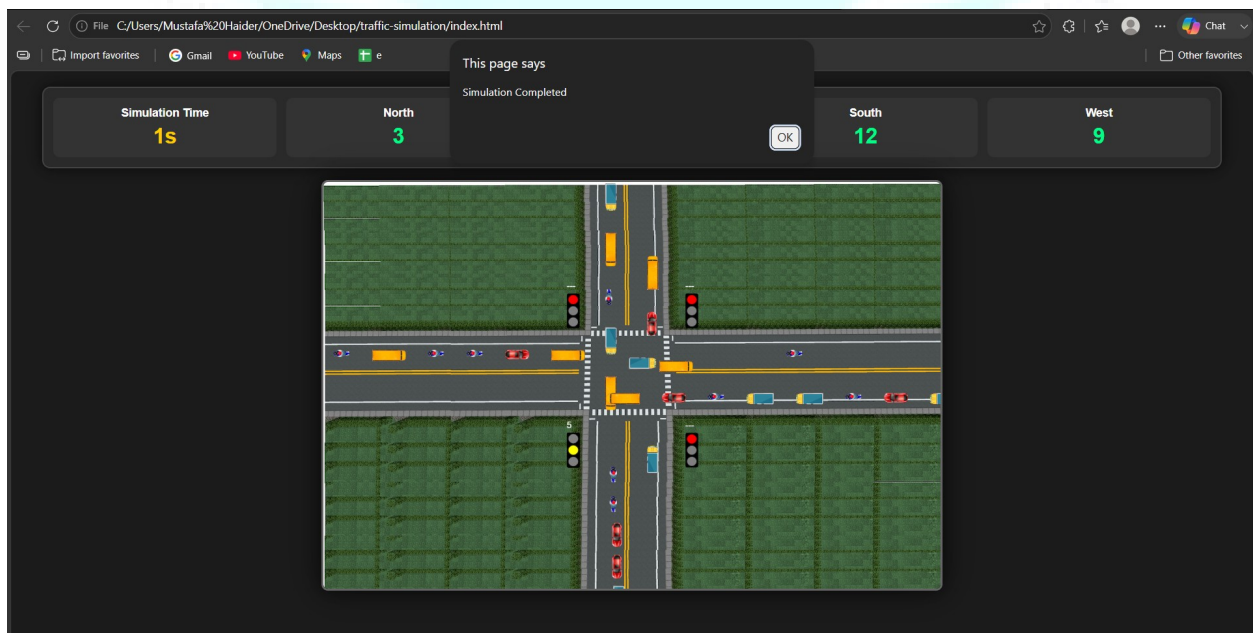


Figure 5.5: Simulation Stops

6. Algorithm / Flowchart

The simulation continuously runs using JavaScript's `requestAnimationFrame()` function. During each frame, vehicles are moved, signals are checked, spacing is maintained and the display is updated.

Algorithm:

1. Load all required images and assets.
2. Initialize the traffic signals and timers.
3. Start the vehicle generation system.
4. Begin the simulation loop.
5. Move vehicles according to their assigned direction.
6. Stop vehicles at the stop line when the signal is red.
7. Allow vehicles to move through the intersection when the signal turns green.
8. Process turning vehicles and update their direction after the turn is completed.
9. Maintain a safe distance between vehicles in the same lane.
10. Remove vehicles that leave the visible area.
11. Update traffic counts and dashboard information.
12. Redraw the entire scene on the canvas.
13. Repeat the process until the simulation timer ends.

Flowchart:

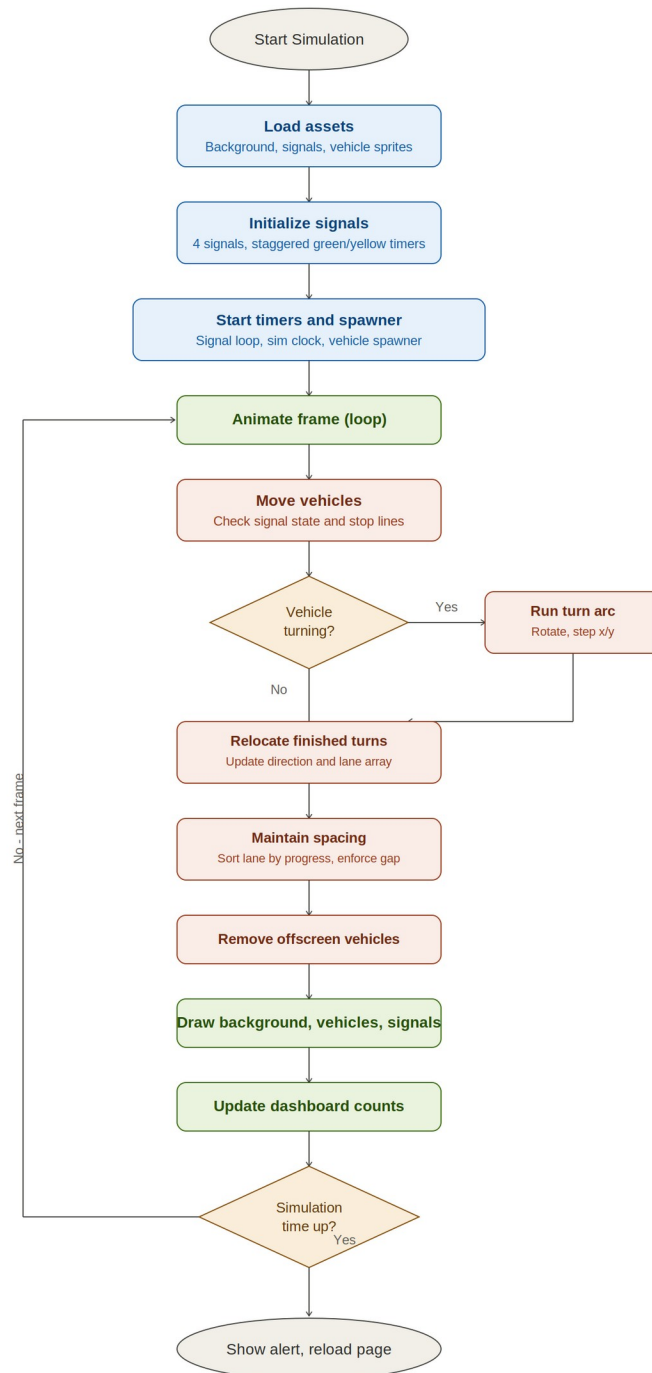


Figure 6.1: Flowchart of the simulation's main animation loop

7. Code Explanation

7.1 Asset loading

All images (background, signal lights and vehicle sprites for each direction and type) are loaded asynchronously with a `loadImage()` function which act as a helper before the simulation starts. This guarantees nothing is drawn until every asset is ready.

```
async function loadAssets() {  
    assets.background = await loadImage("bg/intersection.png");  
    assets.signals.red = await loadImage("signal/red.png");  
    // ...signal/vehicle images loaded the same way  
}
```

7.2 Signal control

The traffic signal system manages four directions. Each signal has a green and yellow phase. Once a signal completes its cycle, control is passed to the next direction.

The green signal duration is adjusted according to the number of waiting vehicles. More vehicles result in a longer green phase within predefined limits.

When a new green phase begins, its duration is recalculated from the number of vehicles currently waiting, within a minimum of 10 and a maximum of 25 seconds.

7.3 Vehicle movement and stop lines

Each Vehicle instance moves along its direction every frame. The `moveRight()`, `moveLeft()`, `moveUp()` and `moveDown()` methods check whether that direction currently has the green light; if not, the vehicle is only allowed to move as far as a fixed stop line coordinate, measured directly from the background image.

7.4 Turning logic

Vehicles flagged will Turn rotate through a 90-degree arc once they reach a turn midpoint near the intersection. The rate of rotation and movement during the turn is scaled to each vehicle's own speed, so bikes turn faster than trucks. Once

the arc completes, the vehicle's direction is updated to the road it has turned into, its position is snapped to that lane's center line and it is moved into the corresponding array in the vehicles object so that spacing and dashboard counts reflect its new lane.

7.5 Spacing between vehicles

To avoid collisions and overlapping, the system maintains a minimum gap between vehicles.

Vehicles in the same lane are sorted according to their position. If two vehicles become too close, the following vehicle is moved slightly back to restore safe spacing.

`maintainVehicleSpacing()` sorts each lane by actual progress along the road, rather than by spawn order and pushes any vehicle that is too close to the one ahead of it backward by a fixed gap.

7.6 Rendering and dashboard

The `animate()` function clears the canvas, draws the background, moves and draws every vehicle, draws the four signal lights with their remaining time, and finally updates the four dashboard counters (North, East, South, West) by reading the length of each direction's vehicle.

During every frame:

- The canvas is cleared.
- The background is drawn.
- Vehicles are updated and displayed.
- Traffic signals are rendered.
- Dashboard statistics are refreshed.

8. Challenges Faced

- **Asset path mismatches:** The background and signal images initially failed to load with 404 errors because the image folders were not nested inside the project folder on disk, but were sitting as sibling folders elsewhere. This was resolved by locating the real file paths and moving the asset folders into the correct project structure.
- **Canvas/background size mismatch:** The canvas element was originally sized differently from the actual background image, which silently stretched the road and made every coordinate used for stop lines and lane positions slightly wrong. This was fixed by measuring the real image dimensions and aligning the canvas size to match exactly.
- **Vehicles driving off the road:** Spawn and lane coordinates were originally estimated rather than measured, causing vehicles to appear on the grass instead of the road. This was corrected by sampling the actual background image pixel by pixel to find the true road and lane boundaries.
- **Vehicles overlapping after turning:** One issue I faced was that vehicles sometimes overlapped after completing a turn. After debugging the code, I found that the vehicle's direction was not being updated correctly after the turn was finished. Because of this, the spacing logic treated it as part of the old lane. I solved the problem by updating the vehicle's direction and lane information immediately after the turning animation completed.
- **Turning vehicles landing off center:** The turning animation moved vehicles by small fixed steps that did not always end exactly on the new lane's center line, causing turned vehicles to overlap straight-lane traffic. This was fixed by snapping a vehicle's position to the correct lane center once its turn was complete.
- **Responsive layout distortion:** An early attempt to make the canvas resize for different screen sizes set conflicting width and height constraints, which stretched the intersection out of proportion. This was resolved by deriving

one dimension purely from the other using a fixed aspect ratio, instead of constraining both independently.

9. GitHub Repository link:

<https://github.com/HareemHaider217/Traffic-Light-Simulation.git>



10. Conclusion

This project demonstrates that a reasonably realistic traffic intersection simulation can be built using only HTML, CSS and JavaScript, without any external simulation engine. Implementing it required careful attention to coordinate geometry, asset organization and timing logic and surfaced several non-obvious bugs particularly around vehicles turning between lanes that needed to be traced back to their root cause rather than patched superficially.

The final simulation models continuous vehicle spawning, adaptive signal timing, turning behaviour and lane spacing, and displays this through a live dashboard. The project reinforced practical skills in DOM, asynchronous asset loading and debugging visual behaviour by reasoning about the underlying data rather than the rendered output alone.

11. References

1. MDN Web Docs: Canvas API
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
2. MDN Web Docs: HTML5 <canvas> element
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/canvas>
3. MDN Web Docs: JavaScript Promises
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
4. MDN Web Docs: Window.requestAnimationFrame()
<https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>
5. Visual Studio Code Live Server extension.
6. Inspiration from:
<https://towardsdatascience.com/traffic-intersection-simulation-using-pygame-689d6bd7687a/>